

A Data Schema and Preprocessing

A.1 De-identified Conversation Logs

We release a de-identified, turn-level log (`conversations_deidentified.json`) together with preprocessing code that maps it to the analysis tables used in this paper. Each row corresponds to a single assistant turn and contains:

- Identifiers: pseudonymous `user_id`, `session_id`, `turn_index`, and `conversation_length` (the within-conversation counter; in the processed tables this appears as `conv_len`).
- Timestamps: `created_at` and (if present) `rated_at`.
- Raw text fields: `user_message` and `assistant_message`, with direct identifiers (names, emails) removed during ingestion; the raw model identifier and all email addresses are dropped. The preprocessing script `load_conversations_json` (in `src/ingest_json.py`) exposes these as `user_text` and `assistant_text` in the analysis tables.
- Tool fields: a raw tool string `tools_used`; the preprocessing script derives a copied column `tool_invoked_raw` and a canonicalized label `tool_invoked` $\in \{\text{none}, \text{web}, \text{code}, \text{calendar}\}$ by mapping heterogeneous tool strings to these buckets.
- Ratings and behavioral metrics: a 1–5 star field rating, latency `response_time` (seconds), message length `message_length` (characters), client-side interaction proxies such as `typing_duration`, and instrumentation fields (`context_tokens`, `context_window_percentage`, `search_enabled`, `tts_played`) where available. In the processed tables these become `rating_star`, `response_time_sec`, and `assistant_tokens`, respectively.

All PII (including email) is removed at ingestion time, and we exclude one participant with insufficient signal from all analyses.

A.2 Sessionization Rules

Given the ingested log, we assign deterministic `session_id` and `turn_index` per user using `conv_len` and time gaps:

- Rows are first sorted by (`user_id`, `created_at`) using a stable sort.
- Within each user, a new session starts when (i) `conv_len`=1, (ii) `conv_len` decreases relative to the previous row, or (iii) there is a large time gap (default 120 minutes) and a jump in `conv_len` that is not $\{\text{prev}, \text{prev}+1\}$.
- The first row per user is always a new session. Subsequent turns increment `turn_index` by one within a session.
- For transparency we additionally record diagnostic flags (e.g., `session_boundary_reason`, `session_boundary_conflict`, and whether `conv_len` equals `turn_index`); summary conflict rates appear in Table 2.

A.3 Derived Feature Tables

From the sessionized log we construct feature tables used for OPE:

- Reward-related fields: objective components R_{tool} and R_{eng} , user satisfaction R_{user} , and an indicator `has_rating`.
- Survey features: per-user eHEALS and GSE scores (Section C), plus median-based bins `eheals_bin`, `gse_bin`.
- Text heuristics: binary flags `contains_number`, `contains_date_rng`, `asks_for_plot`, and `asks_for_citation`, computed from `user_text`.
- Rolling features: for each (`user_id`, `session_id`) we compute lagged means over the previous five turns for tool success rate, latency, and ratings.
- One-hot encodings: current and previous tool and style labels are expanded into one-hot columns (e.g., `tool_web`, `style_detailed`, `prevtool_code`).

B Reward and Rubric Definitions

B.1 Typed Tool-Outcomes and R_{tool}

Table 1 summarizes the rubric we use to label each tool invocation. For reproducibility, we outline the exact mapping:

- **Web Searcher:** we detect success when the assistant response contains at least one URL. Failures are subtyped as `search_no_results` (explicitly “no results” / “unable to find”), `technical_error` (timeouts, exceptions), or `no_url_provided` (tool described but no URL).
- **Code Executor:** we treat the presence of an image or plot markup (Markdown images, HTML `<img>`, or explicit “Figure” tags) as success; failures are `execution_error` (exceptions/tracebacks), `insufficient_data` (empty/no data), or `no_visualization`.
- **Email/Calendar:** confirmations that an email or reminder was sent/scheduled are labeled success; failures are `send_failure`, `invalid_recipient`, or `confirmation_missing`.

Each tool invocation receives a scalar $R_{\text{tool}} \in \{-1, 0, 1\}$: we assign +1 for **success**, -1 for any failure subtype, and 0 when no tool is invoked ($R_{\text{tool}}(\emptyset) \equiv 0$).

B.2 Engagement and User Satisfaction

We define R_{eng} as a small engagement bonus tied to evidence use: when a tool succeeds and the assistant cites at least one URL in the reply, we add $R_{\text{eng}}=0.1$, otherwise $R_{\text{eng}}=0$.

User satisfaction R_{user} is derived from the 1–5 star ratings. For each user i , we compute the within-user z-score

$$R_{\text{user},it} = \frac{r_{it} - \mu_i}{\sigma_i},$$

where r_{it} is the raw rating at turn t , and (μ_i, σ_i) are the mean and standard deviation across all rated turns for user i . If $\sigma_i=0$ (constant ratings) we set $R_{\text{user},it}=0$. We define `has_rating`=1 when r_{it} is observed and 0 otherwise.

B.3 Total Reward Composition

As formalized in Eq. 1, we compose a per-turn scalar utility

$$R_{\text{total},it} = \alpha_i(z_t) R_{\text{user},it} + \beta_i(z_t) R_{\text{tool},it} + \gamma_i(z_t) R_{\text{eng},it},$$

where $(\alpha_i, \beta_i, \gamma_i)$ are pre-specified functions of literacy strata. In the experiments we use fixed weights per literacy bin: for low-literacy users we set $(\alpha_w, \beta_w, \gamma_w) = (0.6, 0.2, 0.2)$, for high-literacy users $(0.3, 0.5, 0.2)$. We report the objective component $R_{\text{obj}} = R_{\text{tool}} + R_{\text{eng}}$ with SNIPS and a literacy-weighted R_{total} that combines R_{obj} and R_{user} as described in the main text.

C Survey Instruments and Archetype Construction

C.1 Health Literacy and Self-Efficacy

Participants completed the eHEALS health-literacy scale [23] and the General Self-Efficacy Scale (GSE) [24] at baseline. For each user we sum the item responses to obtain an eHEALS total score and a GSE total score, and optionally average across multiple survey submissions when available.

To obtain binary strata we compute the median eHEALS and GSE scores over included users and define:

$$\begin{aligned} \text{eheals_bin}_i &= \mathbb{I}\{\text{eHEALS}_i \geq \text{median}(\text{eHEALS})\}, \\ \text{gse_bin}_i &= \mathbb{I}\{\text{GSE}_i \geq \text{median}(\text{GSE})\}. \end{aligned}$$

These bins enter the feature set and are used both for stratified reporting and for defining archetypes.

C.2 Archetype Labels

We define four archetypes by the Cartesian product of literacy and self-efficacy bins:

$$\mathcal{Y} = \{L_{\text{low}} \times E_{\text{low}}, L_{\text{low}} \times E_{\text{high}}, L_{\text{high}} \times E_{\text{low}}, L_{\text{high}} \times E_{\text{high}}\},$$

with the obvious mapping from (eheals_bin, gse_bin) to the four cells. Archetype labels are treated as user-level attributes and are constant across sessions for a given user. In the public code and data release we provide a de-identified per-user JSON file (surveys.json) containing user_id, aggregated eHEALS and GSE scores (eheals_score, gse_score), from which Eheals_bin and gse_bin are constructed.

D Behavior and Rating-Propensity Models

D.1 Behavior Policy Reconstruction

We reconstruct the behavior policy over discrete heads $\{\text{TOOL}, \text{STYLE}\}$ by fitting cross-fitted, calibrated multinomial logistic regressions. Let x_t denote the feature vector at turn t and $a_t^{\text{TOOL}} \in \{\text{none}, \text{web}, \text{code}, \text{calendar}\}$, $a_t^{\text{STYLE}} \in \{\text{concise}, \text{detailed}\}$ the logged actions.

For each head h we:

- Select numeric features from the feature table, excluding obvious label/leakage columns (raw text, rewards, propensities, session identifiers, and downstream OPE outputs).
- Fit a logistic-regression classifier (multinomial, max_iter=1000, balanced class weights) on K folds (up to 5, reduced when classes are rare), using a standard scaler on the feature space.
- For each fold, record out-of-fold class probabilities $\hat{\pi}_b^h(a^h | x_t)$; for binary heads we apply probability calibration (isotonic regression when sample size permits, otherwise Platt scaling).

Assuming conditional independence of heads given x_t , we form the joint propensity as $\hat{\pi}_b(a_t | x_t) = \hat{\pi}_b^{\text{TOOL}}(a_t^{\text{TOOL}} | x_t) \hat{\pi}_b^{\text{STYLE}}(a_t^{\text{STYLE}} | x_t)$. We report per-head Expected Calibration Error (ECE) in Table 2.

D.2 Rating-Propensity Model

To correct for missing ratings we fit a separate logistic-regression model for $m_t = \mathbb{I}\{\text{has_rating}_t = 1\}$ using the same feature table:

- We exclude direct rating-related quantities (e.g., rating_star, R_{user} , R_{tool} , R_{eng}) and downstream OPE outputs from the feature set to avoid target leakage.
- We fit a logistic-regression model with balanced class weights under stratified cross-validation and obtain cross-fitted estimates $\hat{p}_{\text{rate}, t} = \mathbb{P}(m_t=1 | x_t)$, clipped to $[10^{-6}, 1 - 10^{-6}]$.

The overall rating rate and the AUC of the rating-propensity model are reported in Table 2; per-user rating rates are provided in the supplementary tables accompanying the code release.

E Off-Policy Evaluation Estimators

E.1 Importance Sampling and SNIPS

Given a target policy π and behavior policy $\hat{\pi}_b$, we define per-turn importance ratios

$$w_t = \frac{\pi(a_t | x_t)}{\hat{\pi}_b(a_t | x_t)}.$$

The ordinary IPS estimator of expected reward is

$$\hat{V}_{\text{IPS}}(\pi) = \frac{1}{T} \sum_{t=1}^T w_t r_t.$$

To reduce variance we use the self-normalized variant (SNIPS),

$$\hat{V}_{\text{SNIPS}}(\pi) = \frac{\sum_{t=1}^T w_t r_t}{\sum_{t=1}^T w_t},$$

applied to the objective component R_{obj} . In all experiments we clip the raw ratios at $c=50$ before normalization, i.e., we use $w_t = \min\{c, \pi(a_t | x_t) / \hat{\pi}_b(a_t | x_t)\}$.

E.2 Selection-Aware SNIPS and AIPW

When ratings are missing, we distinguish three estimators:

- *Rated-only SNIPS*, which applies SNIPS only over turns with $m_t=1$ and ignores missingness.
- *Selection-aware SNIPS-IPW*, which reweights by the inverse rating propensity:

$$\hat{V}_{\text{SNIPS-IPW}}(\pi) = \frac{\sum_{t=1}^T w_t \frac{m_t}{\hat{p}_{\text{rate},t}} r_t^{\text{user}}}{\sum_{t=1}^T w_t \frac{m_t}{\hat{p}_{\text{rate},t}}}.$$

- *Augmented IPW (AIPW)*, which combines an outcome model $\hat{q}_{\text{user}}(x, a)$ with the selection-weighted residuals, as described in the main text.

We apply AIPW to the user-satisfaction component R_{user} and combine it with SNIPS estimates of R_{tool} and R_{eng} to form literacy-weighted R_{total} .

E.3 Session-Level Bootstrap

To obtain confidence intervals we resample sessions rather than individual turns. Let S index sessions and $v_s(\pi)$ denote a session-level estimate. We:

- Sample S sessions with replacement to form a bootstrap replicate, recompute the policy value, and repeat this procedure $B=1000$ times.
- Report the bootstrap mean and the 2.5/97.5 percentiles as the point estimate and 95% confidence interval in Tables 5 and 6.

This procedure is applied consistently across objective, satisfaction, and total reward metrics.

F Policy Definitions

We evaluate four hand-crafted policies over head-wise distributions $(\pi^{\text{TOOL}}, \pi^{\text{STYLE}})$:

- **NOTOOL**: always selects `tool_none` and a fixed style distribution favoring concise replies ($\pi^{\text{STYLE}} \approx (0.7, 0.3)$).
- **ALWAYSTOOL**: always invokes a tool; if the user message contains numbers or plot cues, the policy chooses code, otherwise web. Style is biased toward detailed responses.
- **HEURISTICGATED**: uses simple text heuristics to decide whether to invoke a tool at all (numbers, date ranges, or explicit plot requests), routing to code for numeric/plot requests and web otherwise; if no such cues are present, it selects none. Style remains moderately concise.
- **PERSONALIZEDWEIGHTS**: implements a lightweight, literacy-aware scoring rule over candidate tools that approximates the weighted reward in Eq. 1, using $(\alpha_w, \beta_w, \gamma_w)$ and simple expectations for R_{user} , R_{tool} , and R_{eng} conditioned on text heuristics. For low-literacy users it favors less tool use and more detailed style; for high-literacy users it increases weight on code and evidence.

All policies factorize over heads; for OPE we evaluate π^{TOOL} and π^{STYLE} at each logged context and gather the probability mass on the logged actions to form importance ratios.

G Simulator Specification

G.1 Personas and Tasks

Our simulator defines four latent personas aligned with the archetypes in Section C. Each persona y is parameterized by (prefers_explanations, tool_tolerance, prefers_detailed, rating_bias), controlling how actions map to ratings. We instantiate three task types:

- **TIMESERIES_PLOT**: requires code plus a visualization flag to succeed.
- **CITE_SOURCE**: requires web search plus a cited URL.
- **CREATE_REMINDER**: requires calendar usage plus a date argument.

Each simulated user episode samples a persona and a task at the start and holds them fixed across turns.

G.2 User Simulator and Typed Rewards

At each turn the policy samples an action over heads, which is then converted into a one-hot tool and style choice. We deterministically check whether the task-specific goal is satisfied to obtain a tool outcome flag, and map $(y, \text{action}, \text{tool outcome})$ into:

- a synthetic star rating (centered around 3.5 plus persona-specific bias), penalizing mismatches such as heavy code usage with non-detailed style for low-literacy personas;
- $R_{\text{tool}} \in \{-1, 1\}$ according to tool success;
- R_{eng} as a small evidence bonus when tools succeed and citations are present;
- R_{user} as a centered version of the rating.

These components follow the same reward composition as in Eq. 1.

G.3 Curiosity and Posterior over Personas

We maintain a categorical posterior $p_t(y)$ over personas, initialized to the uniform distribution. At each turn we update $p_t(y)$ with a simple likelihood function that rewards detailed style and successful tool use differently across archetypes, and compute the entropy drop $H(p_{t-1}) - H(p_t)$ as in Eq. 2. The curiosity bonus $r_t^{\text{curiosity}}$ is applied only in the first few turns (we use $K=2$) with a small coefficient $\lambda \in \{0.05, 0.1, 0.2\}$ and optional exponential decay; the bonus is clipped to keep its magnitude small relative to the task reward. We record trait-identification accuracy and the first turn at which the most probable persona matches the ground truth.

G.4 Simulation Protocol and Metrics

For each policy we simulate multiple users with fixed horizon T and evaluate:

- final return (sum of per-turn rewards in the first rollout);
- goal success rate (task solved at least once);
- pass@3 (fraction of successful runs over $k=3$ repetitions);
- trait-identification accuracy and average turn to identification;
- persona-concordant style rate (fraction of turns where style matches the persona’s detailed vs concise preference).

Table 7 reports summary statistics across policies and curiosity strengths.